



VIRTUE: Virtual Interactive Reality Toolkit for Understanding the EIC

User Guide V 3.2.2

Sean Preins
University of California, Riverside

Introduction

VIRTUE is a standalone customizable event display for collider experiments for desktop or virtual reality (VR). The application is available for free on GitHub, Steam, Google Play, and Zenodo, and features a simplified and engineering model of the ePIC detector for the future Electron-Ion Collider, as well as simulated electron-proton collisions. New collision data and detector geometries can be uploaded into the program as JSON or FBX files within the application folder.

<https://github.com/SeanBP/VIRTUE>

<https://store.steampowered.com/app/2728380/VIRTUE>

https://play.google.com/store/apps/details?id=com.Quantanaut.VIRTUE&hl=en_US

<https://doi.org/10.5281/zenodo.10372110>

System Requirements

The desktop version of the application is available for Windows, macOS, and Linux platforms. System requirements are determined by the Unity 6 runtime requirements:

<https://docs.unity3d.com/6000.5/Documentation/Manual/system-requirements.html>

The VR version of the application is compatible with Oculus Quest 2/3 headsets through Meta Horizon Link and with OpenXR-compatible VR systems, including SteamVR-supported headsets. The host Windows PC must satisfy the same Unity 6 system requirements listed above.

When using Meta Horizon Link with a Quest headset, the additional Meta requirements are:

<https://www.meta.com/help/quest/140991407990979/>

For other OpenXR-compatible headsets, users should consult the requirements of the corresponding OpenXR runtime and VR platform.

The mobile version of the application is available for Android devices. System requirements are determined by the Unity 6 runtime requirements listed above.

The mobile version also includes a VR mode, which splits the display into a stereo pair for use with low-cost mobile VR viewers such as Google Cardboard.

Displaying Event Data

The program comes pre-loaded with a JSON file containing simulated electron-proton data in the ePIC detector at the EIC, as well as an example event data file. To display your own data, create a JSON file within the folder located at:

- **Windows:** "VIRTUE [Platform]_Data/StreamingAssets/Events/"
- **macOS:** Right-click on the application, select "Show Package Contents," and navigate to "Contents/Resources/Data/StreamingAssets/Events/"
- **Linux:** "VIRTUE [Platform]_Data/StreamingAssets/Events/"

The JSON files in this folder contain data for particle collision events within the virtual environment. For best performance, it is recommended to store only a small (< 100) number of events within each file, depending on how populated the events are.

Particle collision data can be generated directly from EDM4eic ROOT files using the Python conversion tools available in the VIRTUE GitHub repository:

<https://github.com/SeanBP/VIRTUE>

The conversion scripts read EDM4eic data products from CERN ROOT files and produce VIRTUE-compatible event JSON files containing tracks, hits, clusters, and other detector information.

The format of the JSON data file is as follows:

```

1 "header": { // Header contains metadata for the event data
2   "version": "3.2.2", // Version of the data format
3   "title": "Example Data", // Optional: Title of the event data
4   "energy_unit": "GeV", // Optional: Unit for energy (default: GeV)
5   "color_bar": "Lin", // Optional: Type of color bar (Lin or Log)
6   "scale": 2.0, // Optional: Scales the size of the event
7   "length_unit": "mm", // Unit of length (e.g., m, cm, mm)
8   "model_file": "Example_Model.json",
9   // Optional: Name of a model file (in the Models folder) to automatically load
   // alongside this event file
10  "time_before": 20, // Optional: Default nanoseconds to animate before the event,
   // applied once when the file loads
11  "time_after": 100, // Optional: Default nanoseconds to animate after the event,
   // applied once when the file loads
12  "speed": 3, // Optional: Default animation speed, applied once when the
   // file loads
13  "particles": [ // Optional: Array of particle objects to be
   // animated before the event
14    {
15      "size": 150.0, // Size of the particle
16      "color_rgba": [1.0, 0.0, 0.0, 1.0], // RGBA color values
17      "ip": [0.0, 0.0, 0.0], // Optional: Interaction point at t=0 (default: origin)
18      "angle_rad": [0.0, 3.14159] // Direction in radians
19    }
20  ],
21  "tracker_settings": {
22    "B_field_T": 2, // Optional: Magnetic field in Tesla (default: 0 T)
23    "B_field_direction": [0.0, 0.0, 1.0],
24    // Optional: Direction of the magnetic field (default: +Z)
25    "segment_ns": 0.05, // Optional: Segmentation length of tracks. Increase
   // this to improve performance.
26    "tracker_boundary": [10000.0, -20000.0, 20000.0]
27    // Cylindrical boundary for the tracker as an array of radius, left boundary,
   // right boundary. All tracks will terminate outside of this boundary.
28  }
29 },
30 // Array of events containing event-specific data
31 "events": [
32   {
33     // Contains metadata for the event
34     "event_data": {
35       "info_text": "Example Event", // Optional: Event description
36       "energy_scale": [0, 10] // Optional: Energy range for the color bar
37     },
38     "hits": [ // Optional: Array of hit objects
39       {
40         "position": [1000, 0, 0], // Position of the hit
41         "time_ns": 10, // Optional: Time of the hit in nanoseconds
42         "size": 200.0, // Size of the hit
43         "color_rgba": [0.5, 0.0, 0.5, 0.8]
44         // Optional: RGBA color values for the hit
45       }
46     ],
47   }
48 }
49
50

```

```

51     "clusters": [                                // Optional: Array of cluster objects
52     {
53         "position": [0, 1000, 1000], // Position of the cluster
54         "granularity": 300,           // Granularity of the cluster
55         "length": 3000,               // Length of the cluster
56         "time_ns": 20,                // Optional: Time in nanoseconds
57         "color_rgba": [0.0, 0.0, 1.0, 1.0]
58         // Optional: RGBA color values for the cluster
59     }
60 ],
61     "jets": [                                    // Optional: Array of jet objects
62     {
63         "length": 3000,                // Length of the jet
64         "R_rad": 0.1,                 // Opening angle in radians
65         "angle_rad": [1.57, 1.57],    // Direction in radians
66         "vertex": [0, 1000, 0],       // Optional: Vertex of the jet (default: origin)
67         "time_ns": 20,                // Optional: Time in nanoseconds
68         "color_rgba": [0.0, 0.0, 1.0, 1.0]
69         // Optional: RGBA color values for the jet
70     }
71 ],
72     "tracks": [                                // Optional: Array of track objects
73     {
74         "qOverP": -3,                 // Optional: Charge over momentum ratio (default
75         : 0)
76         "angle_rad": [2.3, -1.57],    // Direction of the track in radians
77         "vertex": [0.0, 500.0, 0.0], // Optional: Starting position (default: origin)
78         "duration_ns": [5.0, 30.0], // Optional: Start and end times
79         "color_rgba": [0.0, 0.0, 1.0, 1.0]
80         // Optional: RGBA color values for the track
81     }
82 ],
83     "blocks": [                                // Optional: Array of block objects
84     {
85         "position": [1000, 1000, 1000], // Position of the block
86         "time_ns": 20,                  // Optional: Time of the block in
87         nanoseconds
88         "size": [200, 400, 600],        // Size along x, y, z axes
89         "euler_angles_deg": [100, 200, 300],
90         // Optional: Rotation in degrees along x, y, z axes
91         "color_rgba": [1, 0, 0, 1]     // Optional: RGBA color values for the block
92     }
93 ]

```

Displaying 3D Models

VIRTUE comes pre-loaded with a JSON file and FBX files containing the geometry of the ePIC detector at the EIC, as well as an example JSON 3D model file. To display a new 3D model, create a JSON or FBX file in the appropriate directory:

- **Windows:** VIRTUE [Platform] _Data/StreamingAssets/Models/
- **macOS:** Right-click on the application, select "Show Package Contents", and navigate to: Contents/Resources/Data/StreamingAssets/Models/
- **Linux:** VIRTUE [Platform] _Data/StreamingAssets/Models/

For Steam installations, locate the application folder by opening Steam, selecting VIRTUE in the library, clicking the gear icon, then selecting "Manage" → "Browse local files".

FBX Model Support

Many 3D computer-aided design (CAD) programs allow models to be exported in FBX (Filmbox) format. Alternatively, many CAD file formats can be converted to FBX using third-party software, including online file conversion tools.

FBX files in VIRTUE are loaded using the TriLib2 model loading package. To verify compatibility, upload your FBX file to the TriLib2 Model Viewer Web Demo: <https://ricardoreis.net/trilib/trilib2demo/modelviewer/>.

Textures and Materials

An FBX model's textures can be provided in one of two ways, depending on platform.

On the **Desktop**, **VR**, and **Lite** versions, textures do not need to be embedded in the FBX file. Place a folder named `textures` alongside the FBX file, in the same `Models` directory described above, containing the image files (e.g. `.png`, `.jpg`) the model references. VIRTUE will resolve these automatically, matching whatever relative path the file was exported with.

On the **Mobile** version, a new model is instead selected using the "Upload" button, which opens the device's native file picker. Because only the single selected file is transferred into the application, Mobile cannot resolve a separate `textures` folder – FBX files loaded this way must have their textures, materials, and colors embedded directly in the file itself. When exporting an FBX file for use on Mobile, ensure the exporter's "Embed Textures" (or equivalent) option is enabled.

Embedding textures also works on Desktop, VR, and Lite, and is the simpler single-file option there as well if you would rather not manage a separate `textures` folder.

SketchUp FBX Export Procedure

The native FBX exporter in SketchUp often results in improperly rendered models. To ensure proper rendering, it is recommended to convert SketchUp files to FBX using the Unity Editor. The Unity Editor can be downloaded via the Unity Hub at <https://unity.com/download>. To correctly export a SketchUp model to FBX, follow these steps:

1. In SketchUp, **explode** all groups within the model and save.
2. In the Unity Editor, navigate to **Assets** → **Import New Asset...** and select the SketchUp (.skp) file.
3. In the **Assets** folder, create a new folder named **Color**.
4. Select the imported SketchUp file in the **Assets** folder. In the **Inspector** under the **Materials** tab, click "Extract Textures..." and "Extract Materials...", saving them in the **Color** folder.
5. Drag and drop the SketchUp asset into the Unity scene. Adjust its orientation and position as needed.
6. In the **Hierarchy**, right-click the model and select "Export To FBX...".
7. In the export options:
 - Set "Include" to "Model(s) Only".
 - Set "Object(s) Position" to "World Absolute".
 - Ensure "Embed Textures" is checked.
8. The exported FBX file is now ready for use in VIRTUE.

JSON-Based 3D Model Geometry

In addition to supporting FBX models, VIRTUE implements a custom JSON format for defining simplified detector geometries. This format enhances event visibility and allows for individual component labeling. It supports three component types: **toroid**, **block**, and **sphereoid** components.

For optimal visualization, components should be indexed from the innermost to the outermost structures. Circular components should be approximated with no more than 50 sides for improved performance. The expected JSON format follows this structure:

```
1 "header": { // Header contains metadata about the model configuration
2   "version": "3.2.2", // Version of the model format
3   "title": "Example Model", // Optional: Title of the model
4   "length_unit": "m", // Unit of length (e.g., m, cm, mm)
5   "scale": 1.0, // Optional: Scale of the model
6   "event_file": "Example_Events.json"
7   // Optional: Name of an event file (in the Events folder) to automatically load
   // alongside this model
8 },
9 "components": [
10  {
11    "index": 0, // Index of the component to set the render order.
12    "type": "toroid", // Type of the component (toroid)
13    "name": "Example Toroid", // Optional: Name of the toroidal component
14    "position": [0, 0, 0], // Position in space (x, y, z coordinates)
15    "sides": 10, // Number of sides for the toroid
16    "radii": { // Radii for the left and right sides
17      "left": [0, 2], // Inner and outer radii for the left side
18      "right": [1, 2] // Inner and outer radii for the right side
19    },
20    "length": [1, 2], // Inner and outer length
21    "inner_offset": -1 // Optional: Offset of the inner barrel
22    "euler_angles_deg": [10, 20, 45],
23    // Optional: Rotation in degrees along x, y, z axes
24    "color_rgba": [1.0, 1.0, 0.5, 0.3], // RGBA color values
25  },
26  {
27    "index": 1,
28    "type": "block", // Type of the component (block)
29    "name": "Example Block", // Optional: Name of the block component
30    "position": [4, 0, 0], // Position in space (x, y, z coordinates)
31    "size": [1, 2, 3], // Size along x, y, z axes
32    "euler_angles_deg": [0, 45, 0],
33    // Optional: Rotation in degrees along x, y, z axes
34    "color_rgba": [0.5, 1.0, 1.0, 0.3] // RGBA color values
35  },
36  {
37    "index": 2,
38    "type": "sphereoid", // Type of the component (sphereoid)
39    "name": "Example Sphereoid", // Optional: Name of the sphereoid component
40    "position": [8, 0, 0], // Position in space (x, y, z coordinates)
41    "size": [1, 2, 3], // Size along x, y, z axes
42    "euler_angles_deg": [0, 45, 0],
43    // Optional: Rotation in degrees along x, y, z axes
44    "color_rgba": [1.0, 0.5, 1.0, 0.3] // RGBA color values
45  }
46 ]
```

Displaying Tours

The program comes pre-loaded with a JSON file containing "tour" information that will guide the user through different layers of the ePIC detector and collision events. To display your own tour, create a JSON file within the folder located at:

- **Windows:** "VIRTUE [Platform]_Data/StreamingAssets/Tours/"
- **macOS:** Right-click on the application, select "Show Package Contents," and navigate to "Contents/Resources/Data/StreamingAssets/Tours/"
- **Linux:** "VIRTUE [Platform]_Data/StreamingAssets/Tours/"

The JSON files in this folder contain data for scenes in a tour, which references a model and event file. The format of the JSON data file is as follows:

```
1  "header": {
2      "version": "3.2.2",                // Version of the data format
3      "model_file": "Example_Model.json", // Name of the model file
4      "events_file": "Example_Events.json", // Name of the events file
5  }
6  "scenes": [                          // Array of scenes in the tour
7      {
8          "model_settings":
9              "components": [0, 1, 2], // Indexes of the components to show
10             "lines_active": [0, 1], // Indexes of the components to show the
11                                     wireframe of
12         "text":{
13             "title": "Scene 1",                // Title of the scene
14             "body": "This shows a scene with no event." // Body of text to display
15                                     in the scene
16         },
17         "camera_settings":{
18             "position": [ 0.0, 2.0, -10.0 ], // Position of the camera
19             "focus": [ 0.0, 0.0, 0.0 ]      // Position of the focus of the camera
20         },
21     {
22         "model_settings": {
23             "all_components": true // Toggle to display all components in the model
24         }
25         "text":{
26             "title": "Scene 2",
27             "body": "This shows a scene with an event."
28         },
29         "event_settings": {
30             "index": 3,                // Index of the event to animate during the scene
31             "time_before": 20,         // Nanoseconds to animate before the event
32             "speed": 1                 // Speed of the animation
33         },
34         "camera_settings":{
35             "position": [ 0.0, 2.0, -10.0 ],
36             "focus": [ 0.0, 0.0, 0.0 ]
37         }
38     }
39 ]
```

Unity Projects

The Unity project files for both the desktop and VR versions of the application are available on Zenodo, to allow further customization if desired. <https://doi.org/10.5281/zenodo.10372110>

The core rendering and JSON-parsing logic shared across all VIRTUE editions is distributed as a separate local Unity package, `com.quantanaut.virtue.core`, rather than being duplicated inside each project. This shared core folder must also be downloaded and placed alongside the project folder (as a sibling directory, not inside it) before the project will open without errors, since each project's `Packages/manifest.json` references it by a relative path.

To edit the projects, download and unzip the project files together with the shared core folder into the same parent directory, then open the project through the Unity Hub application: <https://unity.com/download>. The way the application renders and animates data from the JSON files can be modified by editing the respective C# scripts in the assets folder of the project, or, for logic shared across all editions, by editing the scripts inside `com.quantanaut.virtue.core`.

Once the desired edits are made to the project, build the project for your desired platform and run it from the executable file this generates. For further details, refer to the Unity documentation: <https://docs.unity.com/>.

Acknowledgments

Developer: Sean Preins

Supervisor: Prof. Kenneth Barish

Electron-Ion Collider: www.bnl.gov/eic/

Simulations and detector design by the ePIC Collaboration:

www.bnl.gov/eic/epic.php

<https://eic.jlab.org/Menagerie/>

Music: Josh Puddington

Logo: Justyna Babinska "JustaSuta"

TriLib 2: Ricardo Reis

<https://ricardoreis.net/trilib-2/>

This work was supported by the University of California, Office of the President.